**2026-10-10 数据库基础**


- 1. 为什么要有数据库?
- 2 主流数据库
- 3. 数据库的基本使用
  - 3.1 什么是数据库?
  - 3.2 数据库的基本使用
  - 3.3 数据库的本质
  - 3.4 服务器、数据库、表关系
- 4. MySQL架构
- 5. SQL分类
- 6. 查看SQL语言的存储引擎
  - 6.1 查看存储引擎
  - 6.2 存储引擎对比


## 1. 为什么要有数据库?

提出问题：我们存储数据使用文件就可以了，为什么还要弄一个数据库?
文件保存数据有以下几个缺点：

- 文件的安全性问题
- 文件不利于数据查询和管理
- 文件不利于存储海量数据
- 文件在程序中的控制很不方便
  一般的文件确实提供了数据的存储功能，但是站在用户的角度文件并没有提供非常好的数据管理能力。
  为了解决文件存储数据所存在的问题，就设计出更加有利于管理数据的东西--数据库，它能更有效的管理数据。数据库的水平是衡量一个程序员水平的重要的指标。
  数据库的存储介质：
- 磁盘
- 内存

## 2 主流数据库

- SQL Server： 微软的产品，.Net程序员的最爱，中大型项目。
- Oracle： 甲骨文产品，适合大型项目，复杂的业务逻辑，并发一般来说不如MySQL。
- MySQL：世界上最受欢迎的数据库，属于甲骨文，并发性好，不适合做复杂的业务。主要用在电商，SNS，论坛。对简单的SQL处理效果好。

- PostgreSQL :加州大学伯克利分校计算机系开发的关系型数据库，不管是私用，商用，还是学术研究使用，可以免费使用，修改和分发。
- SQLite： 是一款轻型的数据库，是遵守ACID的关系型数据库管理系统，它包含在一个相对小的C库中。它的设计目标是嵌入式的，而且目前已经在很多嵌入式产品中使用了它，它占用资源非常的低，在嵌入式设备中，可能只需要几百K的内存就够了。
- H2： 是一个用Java开发的嵌入式数据库，它本身只是一个类库，可以直接嵌入到应用项目中。

## 3. 数据库的基本使用

### 3.1 什么是数据库?

数据库一般指的是，在磁盘或者内存中存储的特定结构组织的数据的一套方案。

> 数据库服务--mysqld
> 数据库--mysql

数据库本质：对数据内容存储的一套解决方案，你给我字段或者要求，我直接给你结果

### 3.2 数据库的基本使用

下面我们来见一见数据库：我们使用mysql建立一个数据库并建立一张表结构，在表结构中插入一些数据--对比一下mysql在Linux中是如何表现的

1. 启动MySQL

```
mysql -uroot -p
```

2. 创建数据库

```
create database helloworld;
```

3. 查看数据库：

```
show databases;
```

> 这里需要注意，创建数据库是创建一个数据库database是单数没有s，查看数据库，也就是查看所有的数据库这里是databases这里是有s的。

4. 创建表结构：

```
create table student(
    id int,
    name varchar(32),
    gender varchar(2)
);
```

```
mysql> use helloworld;
Database changed
mysql> create table student(
    -> name
    -> varchar(32),
    -> age int,
    -> gender varchar(2)
    -> );
Query OK, 0 rows affected (0.01 sec)
```

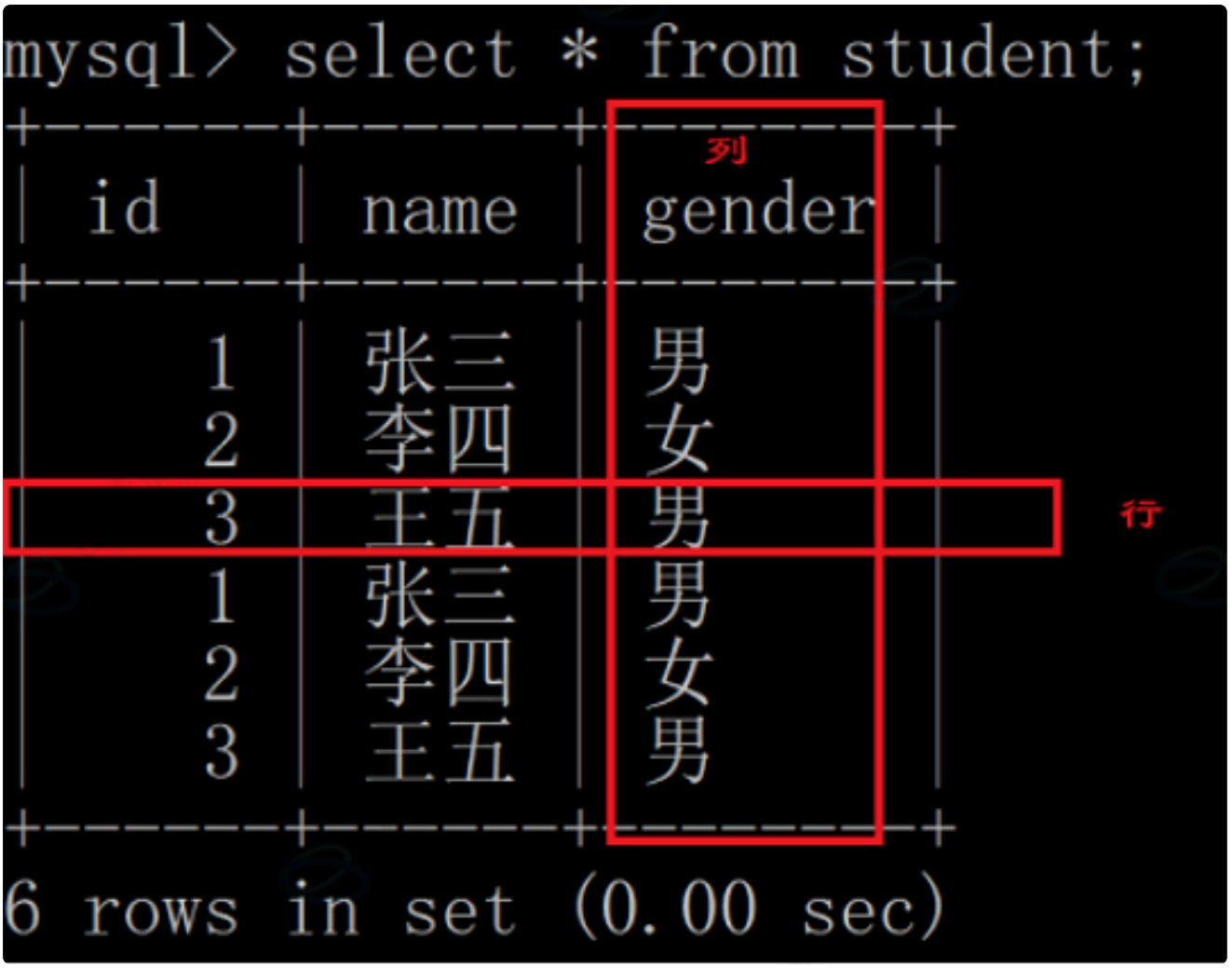
6. 向表中插入数据

```
insert into student (id, name, gender) values (1, '张三', '男');
insert into student (id, name, gender) values (2, '李四', '女');
insert into student (id, name, gender) values (3, '王五', '男');
```

7. 查询表中的数据

```
select * from student;
```

> 这里的“`*`”指的是"所有"的意思




### 3.3 数据库的本质

建立数据库本质就是在Linux下的/var/lib/mysql根目录下，创建一个文件：

```
[root@localhost MySQL]# cd /var/lib/mysql
[root@localhost mysql]# ls
auto.cnf    ca.pem           client-key.pem  ib_buffer_pool  ib_logfile0  ibtmp1  mysql.sock       performance_schema  public_key.pem   server-key.pem
ca-key.pem  client-cert.pem  helloworld      ibdata1         ib_logfile1  mysql   mysql.sock.lock  private_key.pem     server-cert.pem  sys
[root@localhost mysql]# cd helloworld/
[root@localhost helloworld]# ls
db.opt  student.frm  student.ibd
[root@localhost helloworld]# cat db.opt
default-character-set=utf8
default-collation=utf8_general_ci
[root@localhost helloworld]# pwd
/var/lib/mysql/helloworld
[root@localhost helloworld]#
```

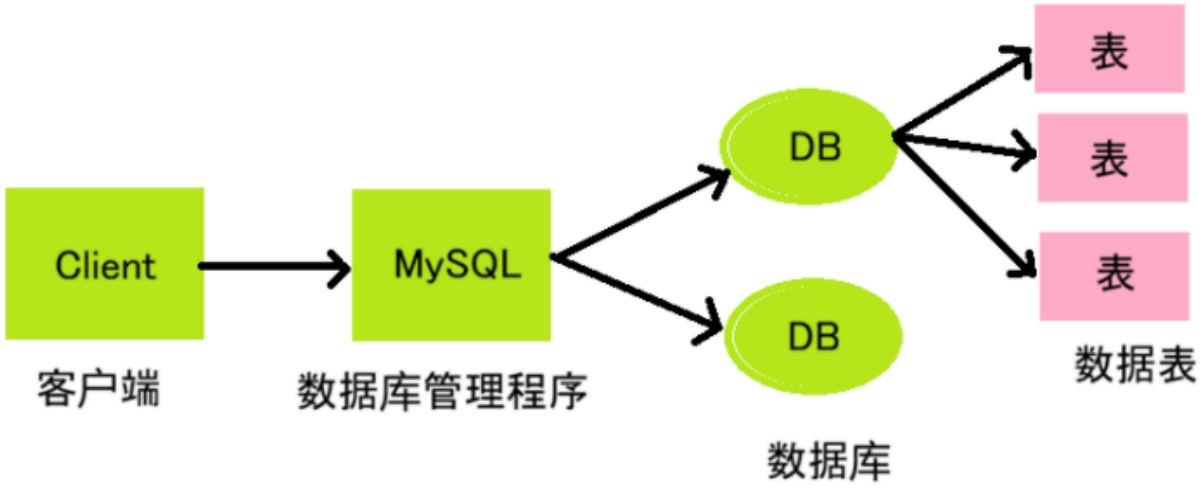
在数据库里建立表，删除表等操作本质上是对文件进行操作

> 这里也对应了Linux下一切皆文件

我们使用创建数据库，查看数据库等工作是谁做的？实际上是mysqld服务帮我做的。
总结一下：
数据库本质其实也是文件，只不过这些文件并不由程序员直接操作，而是由数据库服务帮我们进行操作的，mysql实际上也是操作系统之上的一套文件系统。

### 3.4 服务器、数据库、表关系

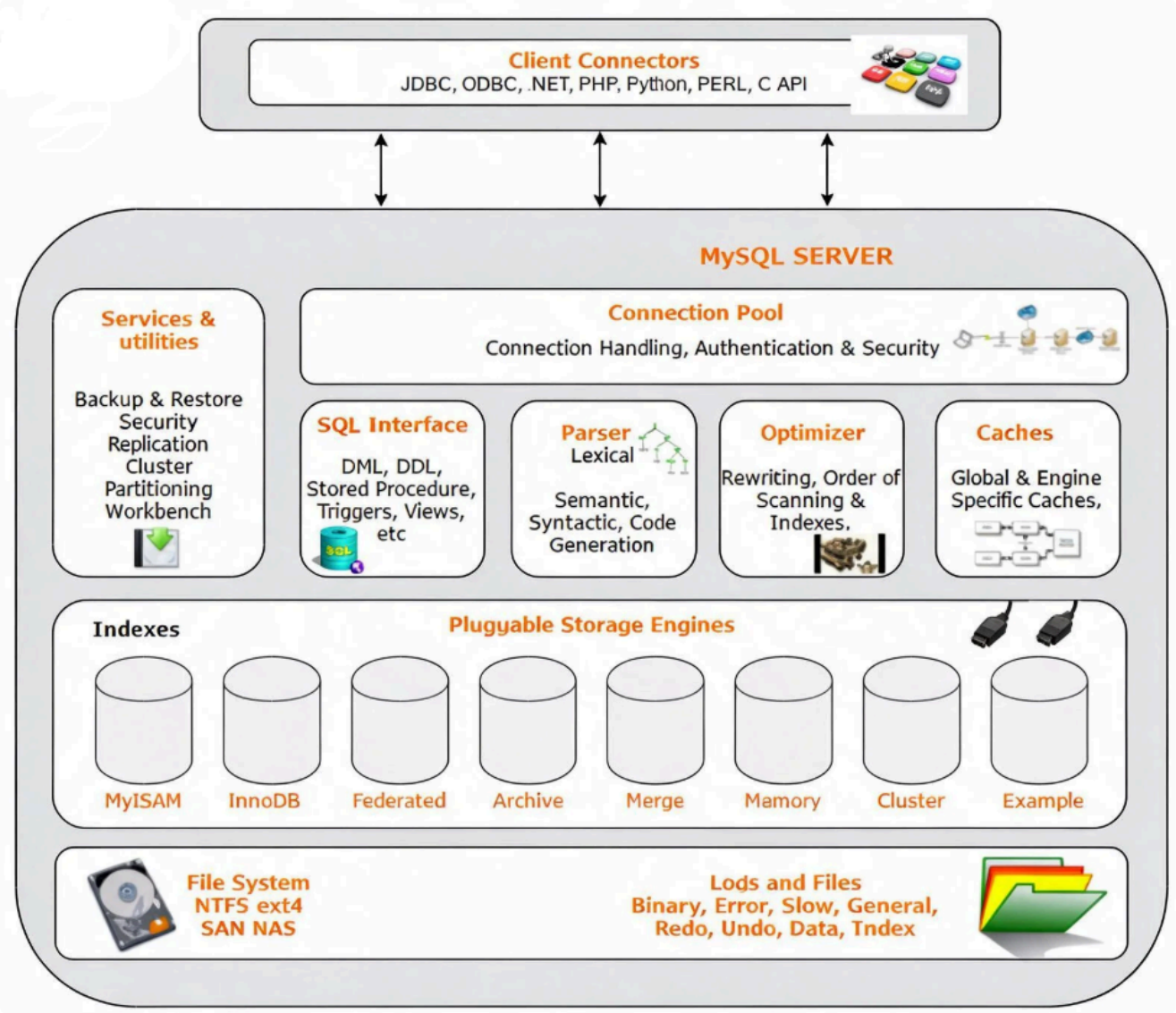
1. 所谓安装数据库服务器，只是在机器上安装了一个数据库管理程序，这个管理程序可以管理多个数据库，一般的开发人员会针对每一个应用创建数据库
2. 为保存应用中实体的数据，一般会在数据库中创建多个表，以保存程序中实体的数据。
3. 数据库服务器，数据库和表的关系如下：




### 4. MySQL架构

MySQLMySQL 是一个可移植的数据库，几乎能在当前所有的操作系统上运行，如 Unix/Linux、Windows、Mac 和 Solaris。各种系统在底层实现方面各有不同，但是 MySQL

基本上能保证在各个平台上的物理体系结构的一致性。




最上层是**客户端连接层**（Client Connectors），负责处理各种编程语言的连接请求；第二层是**MySQL Server 核心层**，包含连接池（处理认证与安全）、SQL 接口（解析 DML/DDL 语句）、解析器（词法与语法分析）、优化器（生成最优执行计划）以及缓存组件；第三层是**可插拔存储引擎层**（如 InnoDB、MyISAM 等），负责底层数据的实际存取；最底层是**文件系统层**，负责将数据、索引及日志（如 Redo Log、Undo Log）持久化到物理磁盘（如 NTFS、ext4）。

## 5. SQL分类

1. DDL数据定义语言(data definition language)：数据定义语言，用来维护存储数据的结构。代表指令：create, drop, alter
2. DML数据操纵语言(data manipulation language): 数据操纵语言，用来对数据进行操作。代表指令： insert，delete，update

> DML中又单独分了一个DQL，数据查询语言，代表指令： select

3. DCL数据控制语言(Data Control Language)：主要负责权限管理和事务代表指令：grant，revoke，commit

## 6. 查看SQL语言的存储引擎

存储引擎是：数据库管理系统如何存储数据、如何为存储的数据建立索引和如何更新、查询数据等技术的实现方法。
**MySQL的核心就是插件式存储引擎，支持多种存储引擎**

### 6.1 查看存储引擎

```
show engines;
```

```
mysql> show engines;
+--------------------+---------+----------------------------------------------------------------+--------------+------+------------+
| Engine             | Support | Comment                                                        | Transactions | XA   | Savepoints |
+--------------------+---------+----------------------------------------------------------------+--------------+------+------------+
| InnoDB             | DEFAULT | Supports transactions, row-level locking, and foreign keys     | YES          | YES  | YES        |
| MRG_MYISAM         | YES     | Collection of identical MyISAM tables                          | NO           | NO   | NO         |
| MEMORY             | YES     | Hash based, stored in memory, useful for temporary tables      | NO           | NO   | NO         |
| BLACKHOLE          | YES     | /dev/null storage engine (anything you write to it disappears) | NO           | NO   | NO         |
| MyISAM             | YES     | MyISAM storage engine                                          | NO           | NO   | NO         |
| CSV                | YES     | CSV storage engine                                             | NO           | NO   | NO         |
| ARCHIVE            | YES     | Archive storage engine                                         | NO           | NO   | NO         |
| PERFORMANCE_SCHEMA | YES     | Performance Schema                                             | NO           | NO   | NO         |
| FEDERATED          | NO      | Federated MySQL storage engine                                 | NULL         | NULL | NULL       |
+--------------------+---------+----------------------------------------------------------------+--------------+------+------------+
9 rows in set (0.00 sec)
```

### 6.2 存储引擎对比

| 特性 (Feature) | MyISAM | BDB | Memory | InnoDB | Archive | NDB |
|---|---|---|---|---|---|---|
| 存储限制 (Storage Limits) | 无 | 无 | 有 | 64TB | 无 | 有 |
| 事务支持 (Transactions: commit, rollback 等) | 不支持 | 支持 | 不支持 | 支持 | 不支持 | 支持 |
| 锁定粒度 (Locking granularity) | 表级 | 页级 | 表级 | 行级 | 行级 | 行级 |
| 多版本并发控制/快照读 (MVCC/Snapshot Read) | 不支持 | 不支持 | 不支持 | 支持 | 不支持 | 支持 |
| 地理空间数据支持 (Geospatial Read) | 支持 | 不支持 | 不支持 | 支持 | 支持 | 支持 |
| B树索引 (B-Tree indexes) | 支持 | 支持 | 支持 | 支持 | 不支持 | 不支持 |
| 哈希索引 (Hash indexes) | 不支持 | 不支持 | 支持 | 不支持 | 不支持 | 不支持 |
| 全文检索索引 (Full text search index) | 支持 | 不支持 | 不支持 | 支持 | 不支持 | 不支持 |
| 聚簇索引 (Clustered index) | 不支持 | 不支持 | 不支持 | 支持 | 不支持 | 不支持 |
| 数据缓存 (Data Caches) | 不支持 | 不支持 | 支持 | 支持 | 不支持 | 支持 |

| 特性 (Feature) | MyISAM | BDB | Memory | InnoDB | Archive | NDB |
| --- | --- | --- | --- | --- | --- | --- |
| **索引缓存 (Index Caches)** | 支持 | 不支持 | 支持 | 支持 | 不支持 | 支持 |
| **数据压缩 (Compressed data)** | 支持 | 不支持 | 不支持 | 支持 | 支持 | 不支持 |
| **数据加密（通过函数） (Encrypted data)** | 支持 | 支持 | 支持 | 支持 | 支持 | 支持 |
| **存储成本（空间占用） (Storage cost)** | 低 (Low) | 低 (Low) | 不适用 (N/A) | 高 (High) | 非常低 (Very Low) | 低 (Low) |
| **内存成本 (Memory cost)** | 低 (Low) | 低 (Low) | 中等 (Medium) | 高 (High) | 低 (Low) | 高 (High) |
| **批量插入速度 (Bulk Insert Speed)** | 高 (High) | 高 (High) | 高 (High) | 低 (Low) | 非常高 (Very High) | 高 (High) |
| **集群数据库支持 (Cluster database support)** | 不支持 | 不支持 | 不支持 | 不支持 | 不支持 | 支持 |
| **复制支持 (Replication support)** | 支持 | 支持 | 支持 | 支持 | 支持 | 支持 |
| **外键支持 (Foreign key support)** | 不支持 | 不支持 | 不支持 | 支持 | 不支持 | 不支持 |
| **备份/时间点恢复 (Backup/Point-in-time recovery)** | 支持 | 支持 | 支持 | 支持 | 支持 | 支持 |
| **查询缓存支持 (Query cache support)** | 支持 | 支持 | 支持 | 支持 | 支持 | 支持 |